IntentFlow – Assistir com Intenção (um "YouTube" criativo com a YouTube Data API v3)

Projeto de portfólio: um aplicativo web que usa a **YouTube Data API v3** e o **player incorporado do YouTube** para oferecer uma experiência de vídeo **oposta ao feed infinito**: o usuário define um **objetivo e um tempo**, o app monta uma **fila curta e finita**, ajuda a **tomar notas ancoradas no tempo do vídeo**, transforma as notas em **cartões de revisão espaçada** e mostra um **painel de atenção** semanal. Documentação completa: conceito, conformidade com as políticas do YouTube, arquitetura, algoritmos, código de referência, testes, roteiro e apresentação de portfólio.

1. Conceito e proposta de valor

Pitch de 30 segundos: "Plataformas de vídeo otimizam o tempo que você passa nelas. O IntentFlow otimiza o que você **leva** delas. Você diz *o que quer aprender e quanto tempo tem*; o app monta uma fila que cabe nesse tempo, você anota enquanto assiste, e ao final a sessão vira revisão e um relatório de atenção."

O que o projeto demonstra no portfólio:

- Integração real com uma API de terceiros sujeita a **cota e políticas** (decisões de arquitetura de verdade).
- **Algoritmos** de verdade: montagem de fila por orçamento de tempo (problema da mochila) e repetição espaçada (SM-2).
- **Full stack** moderno: React, TypeScript, banco relacional, cache, autenticação opcional, testes e deploy.
- **Produto e UX**: tema claro (anti-feed infinito), métricas de atenção, pensamento crítico sobre privacidade.

2. Por que é diferente de um clone comum

Aspecto	Clone tradicional	IntentFlow
Objetivo do app	Maximizar tempo de tela	Maximizar o aprendizado por minuto
Descoberta	Feed infinito e recomendações	**Fila finita** montada por intenção e orçamento de tempo

Aspecto	Clone tradicional	IntentFlow
Interação	Assistir e curtir	**Notas ancoradas no tempo**, capítulos e perguntas de reflexão
Pós-vídeo	Próximo vídeo automático	**Resumo da sessão** e cartões de revisão espaçada
Métricas	Visualizações e curtidas	**Painel de atenção**: tempo por intenção, sessões concluídas, notas e revisões
Custo de API	Alto (muitas buscas)	**Baixo**: usa endpoints de 1 unidade e cache agressivo

> **Importante:** o app **não hospeda nem baixa vídeos**. Ele é um cliente que consulta metadados pela API e reproduz os vídeos **somente pelo player incorporado oficial**, como as políticas do YouTube exigem.

3. Outras ideias de novidade (e por que escolhemos esta)

Ideia	Descrição	Prós	Contras
A – Sessões de Intenção + notas + revisão (escolhida)	Fila por objetivo e tempo, notas no tempo, flashcards e painel de atenção	Muito original, algoritmos reais, baixo custo de cota, ótimo para portfólio	Exige cuidado com políticas (seção 4)
B – Salas de assistir juntos	Reprodução sincronizada e chat em tempo real (WebSocket)	Impressiona em tempo real	Concorrência com apps prontos; mais complexidade de sincronia
C – Mapa de conhecimento	Grafo de vídeos ligados por notas e etiquetas do próprio usuário	Visual e original	Valor depende de muitos dados do usuário
D – Listas colaborativas com votação	Curadoria coletiva de playlists	Social e simples	Menos profundidade técnica

Ideia	Descrição	Prós	Contras
E – Analisador de canais	Painel de estatísticas de canais	Fácil	**Risco de política** (estatísticas e métricas derivadas) e já existe muito

Extensão natural: a ideia B pode virar a v3 do projeto ("sessão em grupo").

4. Conformidade com as políticas do YouTube (leia antes de codar)

O uso da API está sujeito aos **Termos de Serviço dos Serviços de API do YouTube** e às **Políticas para Desenvolvedores**. Elas mudam: confirme sempre na documentação oficial. Resumo das regras relevantes e como o projeto as atende:

Regra (resumo)	Como o IntentFlow cumpre
Não modificar, adicionar ou bloquear a reprodução padrão do player, nem substituir a renderização da plataforma	Player incorporado oficial, **controles intactos**; notas e fila ficam **ao lado**, nunca por cima
Não restringir anúncios que apareceriam normalmente	Nenhum bloqueio ou técnica para evitá-los
Sem reprodução em segundo plano e sem separar áudio ou vídeo; sem download	Nada de modo "áudio" ou "offline"
Exibir **identidade e atribuição do YouTube** e não ocultar a do player	Atribuição visível em cada vídeo e canal; links para o vídeo original
Links para os **Termos de Serviço do YouTube** e a **Política de Privacidade do Google**; política de privacidade própria explicando o uso dos dados do YouTube	Rodapé e página de privacidade
Não armazenar dados da API por mais de 30 dias sem renovar ou apagar	Cache com **TTL curto (horas a poucos dias)** e rotina de expurgo; teste automatizado
Não apresentar dados do YouTube como se fossem outros; **não renomear métricas** do YouTube como métricas próprias	Métricas do YouTube exibidas **como vêm**, com rótulo de fonte; métricas de uso (tempo assistido) rotuladas "no seu histórico"
Cuidado ao combinar dados da API com dados de outras fontes	Dados do usuário (notas, eventos) ficam **em tabelas separadas**; a junção ocorre apenas na exibição

Regra (resumo)	Como o IntentFlow cumpre
Respeitar vídeos **"feitos para crianças"** e limitar dados enviados antes da interação	`autoplay` desligado por padrão; tratar `madeForKids`
Tamanho mínimo do player e **sem sobreposição** que cubra o player ou os controles	Player em tamanho confortável (16:9), sem elementos sobre ele
Usuário controla seus dados; **exclusão em até 30 dias** quando solicitada; não armazenar credenciais	Botão "excluir conta e dados"; login via OAuth, nunca senha
Marca: não sugerir vínculo com o YouTube	Nome próprio, sem usar o logotipo como identidade e com aviso "não afiliado ao YouTube"

4.1 O que NÃO implementar

- **Baixar vídeos**, extrair áudio, "modo podcast" em segundo plano ou **remover anúncios**.
- **Transcrições de vídeos de terceiros** por raspagem ou ferramentas não oficiais (a API só baixa legendas de vídeos dos quais o usuário é proprietário).
- **Raspagem** (*scraping*) do site do YouTube ou uso da API fora do que a política permite.
- Reutilizar a **marca** do YouTube como se fosse sua, ou esconder a atribuição.
- Criar um "score" que **reembale as métricas do YouTube** com outro nome (a prioridade de fila usa dados do **usuário**, como a importância que ele dá a cada canal).

> Para portfólio, o projeto está em modo de testes (sem auditoria). Se um dia for lançado para o público, será preciso solicitar **aumento de cota e auditoria** dos serviços de API do YouTube e, para escopos de OAuth sensíveis, a **verificação do app** pelo Google.

5. Escopo funcional

5.1 Requisitos funcionais

ID	Requisito	Versão
RF-01	Adicionar **canais** por URL, `@handle` ou ID, com validação pela API	MVP

ID	Requisito	Versão
RF-02	Organizar canais em **intenções** (por exemplo, Aprender, Relaxar, Notícias) e definir a **prioridade** de cada canal	MVP
RF-03	**Feed de candidatos**: vídeos recentes dos canais por intenção (via playlist de uploads)	MVP
RF-04	**Iniciar sessão**: objetivo (texto), tempo disponível e intenções	MVP
RF-05	**Fila por orçamento de tempo**: o app seleciona vídeos que cabem no tempo, maximizando a prioridade (problema da mochila)	MVP
RF-06	**Sala de sessão**: player incorporado, fila, cronômetro e painel de notas lado a lado	MVP
RF-07	**Notas ancoradas no tempo** (captura da posição atual do player) com etiquetas; clicar na nota leva o player àquele ponto	MVP
RF-08	**Capítulos** extraídos dos carimbos de tempo da descrição do vídeo, quando existirem	v1.1
RF-09	**Resumo da sessão**: vídeos assistidos, tempo real, notas e pergunta de reflexão ("o que você aprendeu?")	MVP
RF-10	**Cartões de revisão** gerados a partir das notas, com **repetição espaçada (SM-2)**	v1.1
RF-11	**Painel de atenção semanal**: tempo por intenção vs. meta, sessões concluídas, notas, sequência de revisão	v1.1
RF-12	**Modo foco**: sem recomendações nem próximo vídeo automático; ao fim da fila, a sessão termina	MVP
RF-13	Login com **Google** (OAuth) e, opcionalmente, **importar inscrições** do usuário	v2
RF-14	**Exportar** notas e cartões (Markdown ou CSV) e **excluir conta e dados**	v1.1
RF-15	**Sessão em grupo** com reprodução sincronizada e chat	v3 (opcional)

ID	Requisito	Versão
RF-16	Busca por palavra-chave **limitada e com cache** (cota alta)	v1.1

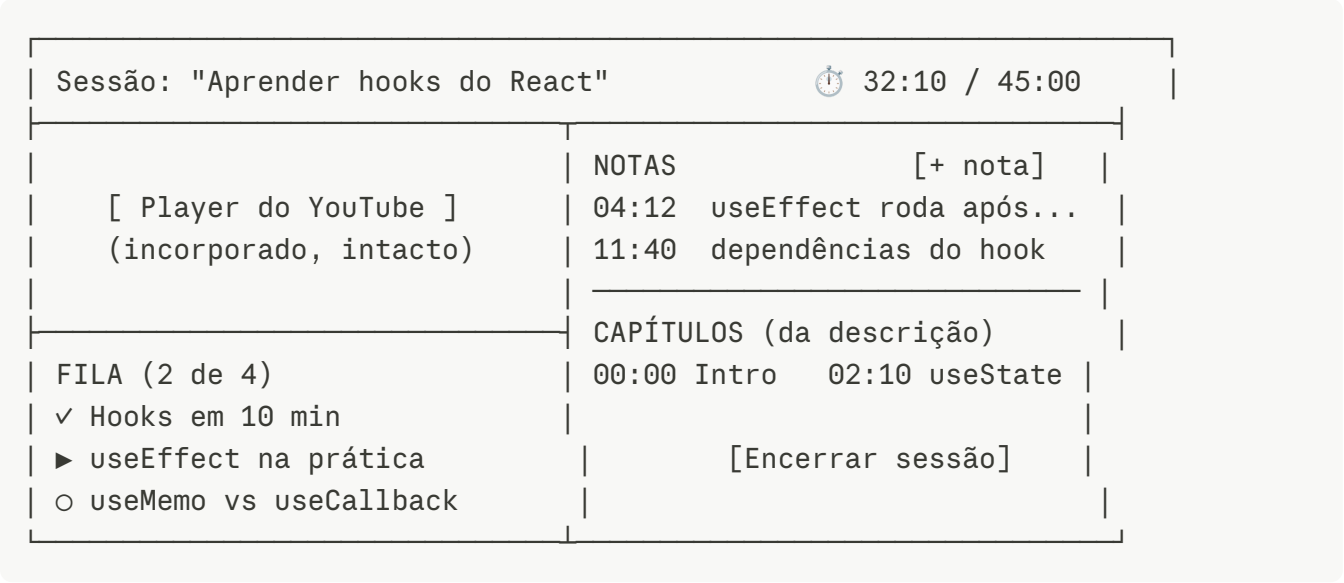
5.2 Requisitos não funcionais

ID	Categoria	Requisito
RNF-01	Conformidade	Cumprir as políticas da seção 4, com testes automatizados para as regras verificáveis (TTL de cache, ausência de sobreposição no player)
RNF-02	Cota	Uso diário da API **abaixo de 30%** do orçamento, mesmo com vários usuários de teste
RNF-03	Desempenho	Páginas principais em menos de 2 s (LCP) com dados em cache
RNF-04	Segurança	Chave da API **somente no servidor**; HTTPS; validação de entrada; proteção OWASP básica
RNF-05	Privacidade	Dados mínimos, exportação e exclusão de conta, política de privacidade clara
RNF-06	Acessibilidade	Navegação por teclado, contraste adequado, rótulos ARIA, legendas respeitadas (não interferir no player)
RNF-07	Resiliência	Quando a cota acabar, o app **degrada com elegância** (dados em cache e aviso), sem quebrar
RNF-08	Qualidade	TypeScript estrito, testes unitários nos algoritmos, E2E dos fluxos críticos, CI
RNF-09	Idioma	Português do Brasil, com textos em arquivo de tradução

6. Experiência e telas

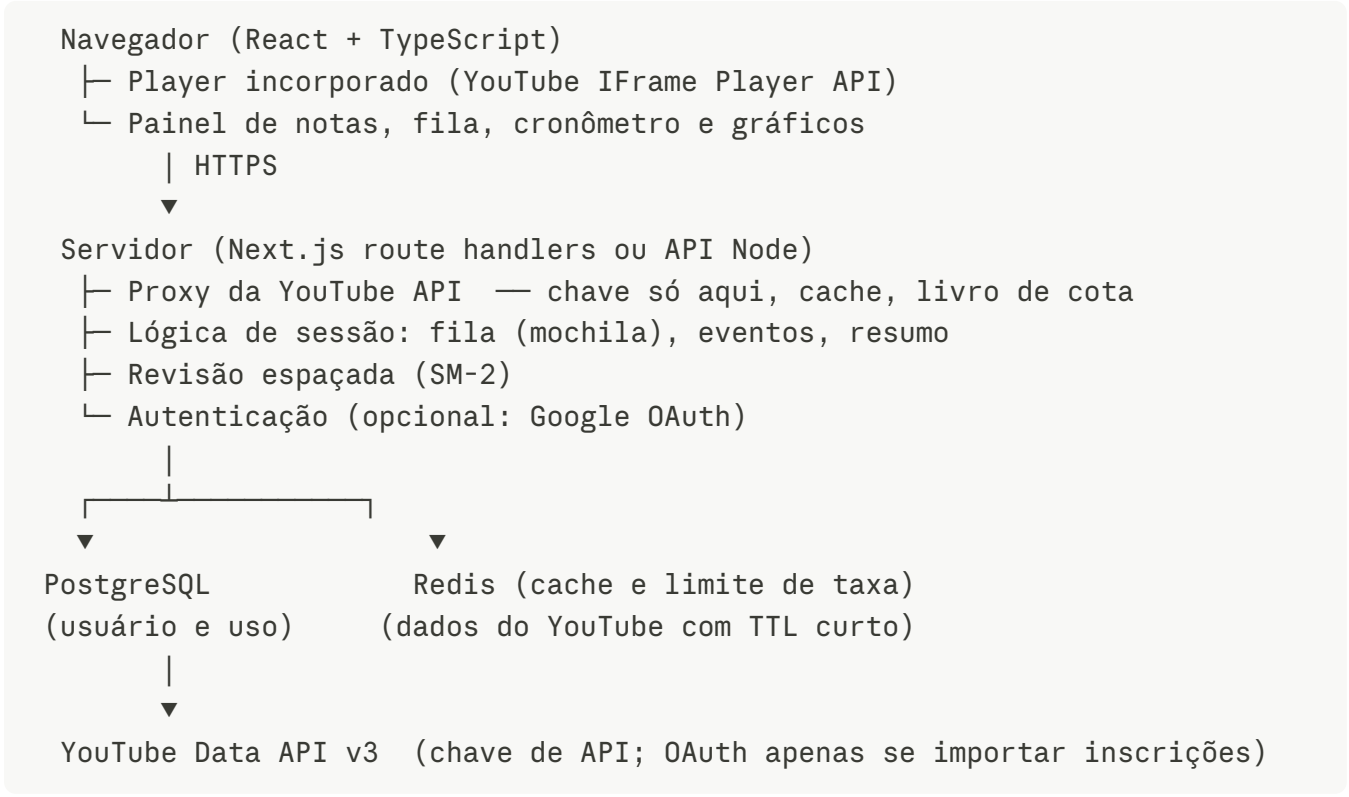
Tela	Conteúdo
Hoje	Sessão em andamento (se houver), cartões de revisão pendentes e atalho "Iniciar sessão"

Tela	Conteúdo
Iniciar sessão	Objetivo, tempo disponível (slider), intenções e prévia da fila sugerida (com possibilidade de trocar um vídeo)
Sala de sessão	Player, fila, cronômetro e notas (wireframe abaixo)
Resumo da sessão	Tempo planejado vs. real, vídeos concluídos, notas e reflexão; botão "gerar cartões"
Revisão	Cartões do dia com autoavaliação (0 a 5)
Painel de atenção	Gráficos semanais de tempo por intenção, sessões, notas e revisões
Canais	Gerenciar canais, intenções e prioridades
Configurações	Metas, privacidade, exportação e exclusão de conta



Princípios de design: calmo, sem contadores de engajamento, sem autoplay do próximo, com fim claro de sessão e destaque para o que o usuário produziu (notas e revisões).

7. Arquitetura e stack



7.1 Stack recomendada

Camada	Escolha	Justificativa
Front-end	**Next.js (App Router) + React + TypeScript**	Rotas de servidor protegem a chave; boa vitrine de portfólio
Estilo	Tailwind CSS + componentes (shadcn/ui)	Velocidade e consistência
Dados no cliente	TanStack Query	Cache e estados de carregamento
Estado local	Zustand	Cronômetro e estado da sessão
Gráficos	Recharts ou Chart.js	Painel de atenção
Player	**YouTube IFrame Player API** (diretamente ou via `react-youtube`)	Única forma permitida de reproduzir
Banco	PostgreSQL com Prisma ou Drizzle	Dados relacionais do usuário
Cache e limites	Redis (por exemplo, Upstash) ou tabela no Postgres	TTL e *rate limit*

Camada	Escolha	Justificativa
Autenticação	Auth.js com Google (v2)	OAuth padrão
Validação	Zod	Contratos de entrada e saída
Testes	Vitest, Testing Library, Playwright	Unitários, componentes e E2E
Deploy	Vercel (ou Docker em VPS) + banco gerenciado	Demonstração pública
Qualidade	ESLint, Prettier, GitHub Actions, Sentry (opcional)	Profissionalismo

8. Estratégia de cota e cache

A cota é a maior restrição técnica do projeto, e **projetar em torno dela é parte do valor de portfólio**.

8.1 Custos relevantes

Método	Uso no app	Custo (unidades)
`channels.list` (por ID ou `forHandle`)	Validar e cadastrar canal; obter playlist de uploads	1
`playlistItems.list`	Feed de vídeos recentes de um canal	1 por página
`videos.list` (até 50 IDs por chamada)	Duração, título, descrição, miniatura, `madeForKids`	1 por chamada
`subscriptions.list` (OAuth)	Importar inscrições (v2)	1 por página
`search.list`	Busca por palavra-chave (opcional)	**100**, a chamada mais cara

> **Confirme os custos e limites atuais** no *Quota Calculator* e na página de cotas do seu projeto no Google Cloud. Por padrão há 10.000 unidades por dia, e a documentação mais recente também descreve um limite próprio para `search.list`. A cota é renovada à meia-noite no horário do Pacífico. Qualquer requisição, mesmo inválida, consome pelo menos 1 unidade.

Decisão de projeto: o app **não depende de** `search.list`. Canais entram por URL, `@handle` ou ID (1 unidade), e o feed vem das **playlists de uploads** (1 unidade por canal). A busca por palavra-chave é um extra com limite por usuário e cache.

8.2 Orçamento de exemplo

Operação	Cálculo	Unidades
Atualizar o feed de 30 canais	30 `playlistItems.list` + 2 `videos.list`	cerca de 32
Atualização a cada 2 horas, durante o dia	12 × 32	cerca de 384 por dia
Cadastro de 10 canais novos	10 `channels.list`	10
20 buscas extras (com cache e limite)	20 × 100	2.000

Com **cache compartilhado entre usuários** (canais comuns são buscados uma vez), o consumo cai bastante.

8.3 Cache e expurgo

Dado	TTL sugerido	Observação
Dados do canal (nome, miniatura, playlist de uploads)	24 horas a 7 dias	Muito estáveis
Lista de uploads recentes	30 a 120 minutos	Atualizada conforme a necessidade
Metadados do vídeo (título, duração, descrição)	1 a 7 dias	**Nunca acima do limite de 30 dias** da política
Estatísticas (se exibidas)	Poucas horas	Exibir como vêm do YouTube

- **Rotina de expurgo** diária que apaga entradas expiradas e **teste automatizado** que falha se houver dado do YouTube mais antigo que o limite.
- **Coalescimento de requisições:** várias requisições simultâneas pelo mesmo recurso compartilham uma única chamada à API.
- **Livro de cota** (`cota_uso`): cada chamada registra data (fuso do Pacífico), método e custo; ao atingir um teto interno (por exemplo, 80% do limite), o servidor responde apenas com cache e avisa o usuário.

- **Identificadores** (como o ID do vídeo) servem de referência para as notas do usuário; confirme na política vigente como tratar IDs e atualize os metadados na exibição.

9. Modelo de dados

Tabela	Campos principais	Observações
`usuarios`	id, nome, e-mail (opcional), criado_em	Conta anônima ou via Google (v2)
`intencoes`	id, usuario_id, nome, meta_minutos_semana	Aprender, Relaxar, Notícias ou personalizadas
`canais`	id, usuario_id, channel_id, intencao_id, prioridade (1 a 5)	**Título e miniatura vêm do cache**, não ficam aqui
`sessoes`	id, usuario_id, objetivo, minutos_planejados, inicio, fim, status, reflexao	Dados do próprio usuário
`sessao_itens`	id, sessao_id, video_id, ordem, status, segundos_assistidos	Só IDs e dados de uso
`eventos_reproducao`	id, sessao_id, video_id, tipo, posicao_s, criado_em	Eventos em lote do player
`notas`	id, usuario_id, video_id, tempo_s, texto, etiquetas	Conteúdo do usuário
`cartoes`	id, nota_id, frente, verso, facilidade, intervalo_dias, repeticoes, proxima_em	Revisão espaçada (SM-2)
`revisoes`	id, cartao_id, qualidade, revisado_em	Histórico de revisões
`cache_youtube`	chave, payload (jsonb), criado_em, expira_em	**Expurgo diário**; nunca acima de 30 dias
`cota_uso`	dia_pt, metodo, unidades	Livro de cota (data no fuso do Pacífico)

Regra de separação: dados do YouTube vivem apenas em `cache_youtube`, com validade curta. Tabelas do usuário guardam **IDs e conteúdo próprio**. Para mostrar o histórico, o app busca os metadados de novo (ou usa o cache) no momento da exibição.

10. API interna do aplicativo

Recurso	Rotas	Descrição
Canais	`GET/POST /api/canais`, `PATCH/DELETE /api/canais/:id`	Cadastro por URL, `@handle` ou ID; prioridade e intenção
Feed	`GET /api/feed?intencao=`	Candidatos recentes (vindo do cache quando possível)
Sessões	`POST /api/sessoes`, `GET /api/sessoes/:id`, `PATCH /api/sessoes/:id/encerrar`	Cria a sessão, devolve a fila e encerra com o resumo
Fila	`POST /api/sessoes/:id/trocar-item`	Troca um vídeo da fila preservando o orçamento
Eventos	`POST /api/sessoes/:id/eventos`	Recebe eventos do player em lote
Notas	`GET/POST /api/notas`, `PATCH/DELETE /api/notas/:id`	Notas ancoradas no tempo
Revisão	`GET /api/revisao/hoje`, `POST /api/revisao/:cartaoId`	Cartões do dia e autoavaliação
Painel	`GET /api/painel/semana`	Agregações da semana
Conta	`GET /api/conta/exportar`, `DELETE /api/conta`	Exportação e exclusão de dados
Operação	`GET /api/saude`, `GET /api/saude/cota`	Saúde e consumo da cota (restrito)

Padrões: validação com Zod, erros em formato único (por exemplo, RFC 7807), paginação e *rate limit* por usuário e por IP.

11. Algoritmos principais

11.1 Fila por orçamento de tempo (problema da mochila)

Dado um conjunto de vídeos candidatos, cada um com **duração** e **valor** (a prioridade que o **usuário** deu ao canal, somada a bônus como "ainda não assistido"), escolher o subconjunto de **maior valor** cujo tempo total **não ultrapasse** os minutos disponíveis. É a mochila 0/1, resolvida por programação dinâmica em O(n × T).

```
type Candidato = { videoId: string; duracaoMin: number; valor: number };

// duracaoMin deve ser inteiro (use Math.ceil sobre os segundos)
export function montarFila(candidatos: Candidato[], orcamentoMin: number):
Candidato[] {
  const n = candidatos.length;
  const dp = Array.from({ length: n + 1 }, () => new Array<number>
(orcamentoMin + 1).fill(0));

  for (let i = 1; i <= n; i++) {
    const { duracaoMin, valor } = candidatos[i - 1];
    for (let t = 0; t <= orcamentoMin; t++) {
      dp[i][t] = dp[i - 1][t];
      if (duracaoMin <= t) {
        dp[i][t] = Math.max(dp[i][t], dp[i - 1][t - duracaoMin] + valor);
      }
    }
  }

  const fila: Candidato[] = [];
  for (let i = n, t = orcamentoMin; i > 0; i--) {
    if (dp[i][t] !== dp[i - 1][t]) {
      fila.push(candidatos[i - 1]);
      t -= candidatos[i - 1].duracaoMin;
    }
  }
  return fila.reverse();
}
```

Detalhes de produto: limitar o número de vídeos (por exemplo, no máximo 6), ordenar a fila por prioridade e deixar o usuário **trocar** itens. **Casos de teste:** orçamento zero, vídeo mais longo que o orçamento, empates, muitos candidatos iguais.

11.2 Duração em ISO 8601

A API devolve a duração como `PT1H2M3S`. Converta para segundos e depois para minutos (arredondando para cima).

```
export function duracaoIsoParaSegundos(iso: string): number {
  const m = /^P(?:(\d+)D)?T?(?:(\d+)H)?(?:(\d+)M)?(?:(\d+)S)?$/.exec(iso);
  if (!m) return 0;
  const [, d = "0", h = "0", min = "0", s = "0"] = m;
  return Number(d) * 86400 + Number(h) * 3600 + Number(min) * 60 + Number(s);
}
```

11.3 Capítulos a partir da descrição

Procurar linhas com carimbos de tempo (`0:00` , `12:34` , `1:02:03`) no início de linha da descrição, exigir ao menos 3 itens e o primeiro em `0:00` . Exibir como **lista navegável ao lado** do player (clicar chama `seekTo`). O texto é do YouTube e permanece dentro do cache com TTL.

11.4 Revisão espaçada (SM-2)

```
export function proximaRevisao(c: Cartao, qualidade: 0 | 1 | 2 | 3 | 4 | 5):
Cartao {
  let { facilidade, intervaloDias, repeticoes } = c;

  if (qualidade < 3) {
    repeticoes = 0;
    intervaloDias = 1;
  } else {
    repeticoes += 1;
    intervaloDias =
      repeticoes === 1 ? 1 : repeticoes === 2 ? 6 : Math.round(intervaloDias
* facilidade);
  }

  facilidade = Math.max(1.3, facilidade + 0.1 - (5 - qualidade) * (0.08 + (5
- qualidade) * 0.02));
  return { ...c, facilidade, intervaloDias, repeticoes, proximaEm:
adicionarDias(new Date(), intervaloDias) };
}
```

Valores iniciais: `facilidade = 2,5` , `intervaloDias = 0` , `repeticoes = 0` .

11.5 Tempo assistido de verdade

Somar segundos **somente enquanto o player está em reprodução** e a aba está visível; enviar os eventos em lote (por exemplo, a cada 15 segundos e ao encerrar). Esse tempo é uma métrica **do app** e deve ser rotulada como "no seu histórico".

12. Integração com a YouTube API e o player

12.1 Cliente de servidor com cache e livro de cota

```
const BASE = "https://www.googleapis.com/youtube/v3";

export async function ytGet<T>(
  recurso: string,
  params: Record<string, string>,
  custo: number,
): Promise<T> {
  const chave = `yt:${recurso}:${JSON.stringify(params)}`;
  const emCache = await cache.get<T>(chave);
  if (emCache) return emCache;

  await cota.reservar(recurso, custo); // lança erro se o teto diário interno
foi atingido

  const url = new URL(`${BASE}/${recurso}`);
  for (const [k, v] of Object.entries({ ...params, key:
process.env.YOUTUBE_API_KEY! })) {
    url.searchParams.set(k, v);
  }

  const resp = await fetch(url); // nunca registre a URL: ela contém a chave
  if (!resp.ok) throw new YouTubeApiError(resp.status);

  const dados = (await resp.json()) as T;
  await cache.set(chave, dados, { ttlSegundos: TTL[recurso] ?? 3600 }); //
TTL bem abaixo de 30 dias
  return dados;
}
```

12.2 Feed barato de um canal

1. `channels.list` com `part=contentDetails` devolve `relatedPlaylists.uploads` (1 unidade; guarde no cache).
2. `playlistItems.list` com essa playlist devolve os uploads recentes (1 unidade).
3. `videos.list` com até 50 IDs devolve duração, descrição e `status.madeForKids` (1 unidade).

Total por canal: cerca de 2 unidades, contra 100 de uma busca.

12.3 Player incorporado com eventos

```
"use client";
import { useEffect, useRef } from "react";

function carregarApi(): Promise<void> {
  return new Promise((resolve) => {
    if ((window as any).YT?.Player) return resolve();
    const s = document.createElement("script");
    s.src = "https://www.youtube.com/iframe_api";
    (window as any).onYouTubeIframeAPIReady = () => resolve();
    document.head.appendChild(s);
  });
}

export function Player({ videoId, onEstado }: {
  videoId: string;
  onEstado: (estado: number, posicaoS: number) => void;
}) {
  const alvo = useRef<HTMLDivElement>(null);
  const player = useRef<any>(null);

  useEffect(() => {
    let ativo = true;
    carregarApi().then(() => {
      if (!ativo || !alvo.current) return;
      player.current = new (window as any).YT.Player(alvo.current, {
        videoId,
        playerVars: { autoplay: 0, playsinline: 1 },   // sem autoplay por
padrão
        events: {
          onStateChange: (e: { data: number }) =>
            onEstado(e.data, player.current.getCurrentTime()),
        },
      });
    });
    return () => { ativo = false; player.current?.destroy?.(); };
  }, [videoId]);

  return <div ref={alvo} className="aspect-video w-full" />;
}
```

Estados do player: `0` encerrado, `1` reproduzindo, `2` pausado. Para criar uma nota, leia `getCurrentTime()` e salve o tempo; para voltar ao ponto de uma nota, use `seekTo(tempo, true)`. **Não** coloque elementos por cima do player.

13. Segurança e privacidade

- **Chave da API somente no servidor**, em variável de ambiente, e **restrita** no Google Cloud (apenas YouTube Data API v3 e, se possível, por IP). Nunca no código do navegador nem no repositório.
- **OAuth (v2):** pedir somente o escopo necessário (leitura de inscrições), tratar revogação e não armazenar credenciais. Em modo de teste, o app mostra aviso de não verificado, aceitável para portfólio.
- **Dados mínimos:** sem rastreadores de terceiros; sem enviar dados do YouTube a outros serviços.
- **Privacidade:** política clara citando o uso dos Serviços de API do YouTube, links para os Termos do YouTube e para a Política de Privacidade do Google, e instruções para revogar o acesso nas configurações de segurança do Google.
- **Direitos do usuário:** exportar (Markdown e JSON) e **excluir conta e dados** (inclusive tokens), dentro do prazo exigido.
- **Crianças:** tratar vídeos `madeForKids` conforme as regras e evitar autoplay.
- **Aplicação:** *rate limit*, validação com Zod, cabeçalhos de segurança, proteção contra CSRF e XSS (as notas são texto do usuário e devem ser sanitizadas ao exibir).

14. Testes e qualidade

Tipo	Escopo	Ferramentas
Unitários	`montarFila`, `duracaoIsoParaSegundos`, SM-2, parser de capítulos, cálculo de tempo assistido	Vitest
Contrato da API	Respostas gravadas (fixtures) da YouTube API, tratamento de 403 de cota e de IDs inexistentes	Vitest com mocks HTTP
Integração	Rotas internas com Postgres em container, cache e livro de cota	Supertest e Testcontainers
Conformidade	Nenhum dado do YouTube com idade acima do limite; ausência de elementos sobrepostos ao player; atribuição presente	Testes automatizados e verificação visual
E2E	Cadastrar canal, iniciar sessão, criar nota, encerrar, revisar cartão	Playwright (com a API mockada)
Acessibilidade	Navegação por teclado e contraste	axe-core e Lighthouse

Tipo	Escopo	Ferramentas
Cota	Simular teto atingido e verificar a degradação elegante	Teste de integração

Boas práticas: fixtures para nunca gastar cota em testes, modo "demonstração" com dados em cache pré-carregados e metas de cobertura nos algoritmos (por exemplo, acima de 90%).

15. DevOps e operação

- **CI (GitHub Actions):** lint, tipos, testes e build a cada *pull request*.
- **Ambientes:** desenvolvimento local (Docker Compose com Postgres e Redis), *preview* por PR e produção.
- **Deploy:** Vercel (front e rotas) + banco e Redis gerenciados; segredos no painel do provedor.
- **Tarefas agendadas:** expurgo diário do cache, atualização do feed em lote e consolidação do painel semanal.
- **Observabilidade:** logs estruturados sem segredos, página de consumo de cota e alerta ao passar de 80% do orçamento.
- **Banco:** migrações versionadas e *backup* diário.

16. Roteiro de implementação

Fase	Entrega	Critério de aceite	Estimativa (meio período)
0 – Fundação	Repositório, CI, Next.js, banco, **teste de fumaça da chave** e leitura das políticas	`channels.list` responde pelo servidor; ADRs iniciais escritos	1 a 2 dias
1 – Núcleo da API	Cliente `ytGet` com cache, livro de cota e expurgo; cadastro de canais; feed por uploads	Cache e cota funcionando; teste de conformidade do TTL verde	1 semana
2 – Sessões	Iniciar sessão, fila por mochila, sala com player, cronômetro e eventos	Sessão completa de ponta a ponta com tempo real medido	1 semana

Fase	Entrega	Critério de aceite	Estimativa (meio período)
3 – Notas e revisão	Notas ancoradas no tempo, capítulos, resumo da sessão, cartões e SM-2	Fluxo nota → cartão → revisão funcionando com testes	1 semana
4 – Painel e privacidade	Painel de atenção, metas, exportação e exclusão de conta, política de privacidade	Painel correto com dados de teste; exclusão apaga tudo	1 semana
5 – OAuth (opcional)	Login com Google e importação de inscrições	Importação sem exceder a cota; revogação tratada	3 a 5 dias
6 – Polimento e portfólio	Acessibilidade, desempenho, testes E2E, demo pública, README e estudo de caso	Lighthouse alto, CI verde e demo estável	1 semana

Total aproximado: 5 a 6 semanas em meio período (cerca de 100 a 140 horas), dependendo da experiência com Next.js e testes.

17. Como apresentar no portfólio

17.1 README (estrutura sugerida)

- Título, **pitch**, captura de tela ou GIF da sala de sessão e link da **demo**.
- **Problema e hipótese:** por que um app contra o feed infinito.
- **Funcionalidades** e **decisões de arquitetura** (diagrama da seção 7).
- **Cota e conformidade:** como o projeto vive dentro dos limites (tabela de cota e regras das políticas).
- **Algoritmos:** mochila e SM-2, com explicações curtas e testes.
- **Como executar** (variáveis de ambiente, `docker compose up`, `npm test`).
- **Aviso:** "Projeto de estudo, não afiliado ao YouTube" e links para os termos.

17.2 Estudo de caso (página ou artigo)

Problema → decisão de produto → restrições (cota e políticas) → arquitetura → algoritmos → resultados (consumo de cota, cobertura de testes, desempenho) → o que faria diferente.

17.3 ADRs sugeridos

- ADR-001: não depender de `search.list`.
- ADR-002: política de cache e limite de 30 dias.
- ADR-003: fila por programação dinâmica (mochila).
- ADR-004: OAuth fora do MVP.
- ADR-005: Next.js com rotas de servidor em vez de API separada.
- ADR-006: como e quando registrar eventos de reprodução.

17.4 Demo

- **Modo demonstração** com canais e vídeos pré-carregados no cache, para não depender de cota durante a apresentação.
- Vídeo de 2 a 3 minutos: criar sessão, assistir, anotar, encerrar, revisar cartões e abrir o painel.

18. Riscos

Risco	Prob.	Impacto	Mitigação
Cota esgotada durante a demo	Média	Alto	Modo demonstração, cache compartilhado, teto interno de 80%
Violar uma política (sobreposição no player, cache acima de 30 dias)	Média	Alto	Testes de conformidade e revisão da seção 4 a cada mudança
Mudanças na API, nos custos ou nas políticas	Média	Médio	Camada de acesso isolada, leitura periódica da documentação e dos avisos
Interpretação de "modificar a reprodução padrão" (fila sem próximo automático)	Baixa	Médio	Manter controles do player intactos; a fila é lógica do app, e não do player
Chave da API vazada	Baixa	Alto	Somente servidor, restrição da chave e segredo fora do Git
Escopo grande demais	Alta	Médio	MVP enxuto (fases 0 a 3) e extensões depois
Uso de dados combinados fora do permitido	Baixa	Médio	Separação de tabelas e junção só na exibição

Risco	Prob.	Impacto	Mitigação
Falta de dados na demo (canais sem vídeos recentes)	Média	Baixo	Canais de exemplo estáveis e *fixtures*

19. Extensões futuras

- **Sessão em grupo:** salas com reprodução sincronizada pelos eventos do IFrame Player API e chat em tempo real (WebSocket), com revisão cuidadosa das políticas.
- **Perguntas de revisão por IA** geradas **apenas a partir das notas do próprio usuário** (e não do conteúdo do YouTube), com opção de desligar.
- **PWA e notificações** de revisão do dia.
- **Integração com calendário** para reservar blocos de estudo.
- **Mapa de conhecimento:** grafo de notas e etiquetas do usuário.
- **Exportação para ferramentas de notas** (Markdown, Obsidian, Notion).
- **Internacionalização** com novos idiomas.

20. Referências

- Documentação da **YouTube Data API v3** (visão geral, *Quota Calculator*, referência de métodos e formulário de extensão de cota).
- **Termos de Serviço dos Serviços de API do YouTube** e **Políticas para Desenvolvedores**, incluindo *Required Minimum Functionality* e o histórico de revisões.
- Referência do **YouTube IFrame Player API** e parâmetros do player incorporado.
- Documentação de **OAuth 2.0 do Google** e da verificação de aplicativos.
- Wozniak, P. (1990) — algoritmo **SM-2** (SuperMemo) para repetição espaçada.
- Cormen, T. et al. — *Introduction to Algorithms* (programação dinâmica e problema da mochila).
- **LGPD** (Lei 13.709/2018) e orientações da ANPD.
- Documentação de Next.js, Prisma ou Drizzle, TanStack Query, Vitest e Playwright.

21. Próximos passos

1. **Proteger a chave:** no Google Cloud, restrinja-a à YouTube Data API v3 (e por IP, se possível) e guarde-a em variável de ambiente no servidor.
2. **Teste de fumaça** da chave e verificação da cota atual no console:

```
curl "https://www.googleapis.com/youtube/v3/channels?
part=snippet,contentDetails&forHandle=@GoogleDevelopers&key=$YOUTUBE_API_KEY"
```

3. **Ler as políticas** (seção 4) e registrar as decisões nos ADRs iniciais.
4. **Criar o repositório** com CI e implementar **primeiro** o cliente `ytGet` com cache, livro de cota e teste de conformidade, antes de qualquer tela.
5. Seguir as fases 1 a 3 do roteiro e só depois decidir sobre OAuth, painel e extensões.